

Project Report: GPU-Accelerated SSSP Optimization via Three-Tier Hybrid Edge Sorting

Topic: GPU Heterogeneous Parallel Sorting Optimization for Large-Scale Graph Processing

Base Framework: ADDS (Asynchronous Dynamic Delta-Stepping)

Optimization Scheme: PRO (Property-driven Reordering)

You can refer to my reading project report to learn more about ADDS.

1. Introduction

The Single-Source Shortest Path (SSSP) problem is a fundamental challenge in graph theory and parallel computing. Given a graph $G = (V, E)$ with non-negative edge weights and a source vertex s , the goal is to find the shortest path from s to all other vertices. While classical algorithms like Dijkstra's and Bellman-Ford provide the theoretical foundation, modern large-scale applications—ranging from social network analysis to real-time navigation—demand high-performance implementations on massively parallel architectures like GPUs.

One of the most advanced GPU-based SSSP frameworks is ADDS (Asynchronous Dynamic Delta-Stepping). ADDS utilizes complex asynchronous optimizations to minimize synchronization overhead. However, ADDS, like many other SSSP implementations, encounters significant performance degradation when processing power-law graphs. Power-law graphs are characterized by a highly skewed degree distribution, where a small number of vertices (hubs) have an extremely high number of edges. This skewness leads to load imbalance and poor cache locality.

This project implements the PRO (Property-driven Reordering) optimization within the ADDS framework. The core idea of PRO is to reorder nodes and sort outgoing edges based on graph properties to improve execution efficiency. The primary technical challenge addressed in this project is the development of a Three-Tier Hybrid Strategy for large-scale graph edge sorting on GPUs. This report details the design, implementation, and performance evaluation of this sorting scheme and its impact on SSSP execution.

2. Background and Motivation

2.1 The CSR Format

Large-scale sparse graphs are typically stored in the Compressed Sparse Row (CSR) format. CSR consists of three arrays:

- Vertex Array: Stores the starting index of each vertex's outgoing edges.

- Edge Array: Stores the destination node IDs of the edges.
- Weight Array: Stores the corresponding edge weights.

While CSR is space-efficient, it does not inherently provide any ordering of edges within a vertex’s neighbor list. In traditional SSSP, the order in which edges are relaxed does not affect correctness but significantly impacts performance.

2.2 The ADDS Baseline

ADDS improves upon the classic Delta-Stepping algorithm by removing global synchronization barriers. It uses a Manager-Worker thread block model to dynamically adjust the Δ parameter and manage multiple priority buckets asynchronously. Despite its efficiency, the “arrival order” of updates in ADDS is somewhat random. In power-law graphs, if the edges leading to the actual shortest path are processed late in the iteration, many redundant edge relaxations occur, wasting GPU compute cycles.

2.3 The PRO Optimization Concept

The PRO (Property-driven Reordering) scheme consists of two main reordering strategies:

- Node Renumbering: High-degree nodes are accessed more frequently in almost all graph algorithms. By renumbering nodes in descending order of their degree, we can ensure that high-degree “hubs” have lower IDs, which can lead to better spatial locality in vertex-indexed arrays.
- Edge Sorting: For SSSP, edges with smaller weights are more likely to lie on the optimal shortest path. By sorting each node’s outgoing edges in ascending order of weight, the algorithm is more likely to find a “near-optimal” distance earlier, allowing it to prune or ignore subsequent updates with larger weights (a greedy heuristic).

Sorting edges for every single node in a graph with millions of vertices is a massive preprocessing task. Standard GPU sorting libraries like Thrust often struggle with “segmented sorting” when the segments (neighbor lists) vary in size from 1 to over 100,000.

3. Challenges in Parallel Edge Sorting

Implementing an efficient segmented sort for graph edges on a GPU presents several technical hurdles:

- Skewed Distributions: In power-law graphs, most nodes have very few edges (e.g., degree < 32), while a few nodes have thousands. A “one-size-fits-all” kernel will either suffer from massive warp divergence (if optimized for large segments) or be unable to handle large segments (if optimized for small ones).

- **Multi-Key Sorting:** The sorting criteria is not just the weight. To ensure stability and further improve locality, we must sort by (Weight, Destination Node ID). This increases the complexity of the comparison function and the amount of data moved per swap.
- **Memory Hierarchy Bottlenecks:** Standard comparison-based sorts like Bitonic Sort or Merge Sort require frequent data exchanges. Moving data between registers, shared memory, and global memory must be carefully orchestrated to maximize bandwidth.
- **Complexity vs. Performance:** While a global `thrust::sort_by_key` approach is possible, it requires creating massive “segment ID” arrays, which can exceed GPU memory limits for large graphs. Also, its theoretical time complexity is suboptimal. A per-node approach is more memory-efficient but harder to parallelize effectively.

4. The Three-Tier Hybrid Strategy

To address these challenges, I designed a Three-Tier Hybrid Strategy that selects the optimal sorting algorithm and storage location based on the degree of the node being processed.

4.1 Tier 1: Warp Shuffle Sorting (Degree ≤ 32)

For nodes with a degree of 32 or less, the entire neighbor list can be handled by a single Warp (32 threads).

- **Storage:** Data is kept entirely in Registers.
- **Technique:** I use Warp Shuffle Instructions (`__shfl_xor_sync`) to perform a Bitonic Sort.
- **Advantages:** This approach involves zero-latency data exchange because it avoids the shared memory load/store overhead. Synchronization is implicit within the warp. This tier processes the vast majority of nodes in a power-law graph with extremely high throughput.

4.2 Tier 2: Shared Memory Sorting (Degree 33 – 2048)

Nodes with intermediate degrees cannot fit in registers but can fit within the Shared Memory of a thread block.

- **Storage:** Neighbor lists are batch-loaded into Shared Memory.
- **Technique:** A block-level Bitonic Sort is used. Each thread handles multiple elements if the degree exceeds the block size.
- **Issue & Resolution:** Segment sizes are rarely powers of two. While padding with `INT_MAX` to the next power of two is a common strategy, my tests

indicated that boundary checking performed slightly better for the diverse degree ranges found in real-world graphs.

4.3 Tier 3: Global Memory Sorting (Degree > 2048)

Hub nodes in power-law graphs can have tens of thousands of edges, far exceeding shared memory capacity.

- Storage: Global Memory.
- Technique: A hybrid optimization strategy is used. Large-scale steps of the sort (where the comparison distance j is large) are performed directly in Global Memory. Once the segment is partitioned into smaller chunks that fit into shared memory (small j values), they are batch-loaded into shared memory for high-speed processing.
- Advantages: This reduces the total number of global memory transactions and ensures that all memory accesses are naturally aligned and coalesced.

4.4 The Edge Comparison Function

To maintain the stability of the sort and optimize the SSSP process, the following comparison logic was implemented as a `__device__` function: code C++

```
__device__ __forceinline__
bool edge_less_than(edge_data_type w1, index_type d1,
                    edge_data_type w2, index_type d2) {
    if (w1 != w2) return w1 < w2; // Primary Key: Weight
    return d1 < d2;               // Secondary Key: Destination ID
}
```

5. Implementation Details

5.1 CUDA Kernel Design

The sorting is executed as a series of kernels, where nodes are assigned to different tiers based on their degree. This prevents “heavy” nodes from stalling “light” nodes within the same warp or block.

- Tier 1 Kernel: Uses one warp per node. High occupancy is achieved because of low resource usage.
- Tier 2 Kernel: Uses one thread block per node. Shared memory is dynamically allocated based on the maximum degree in the batch.
- Tier 3 Kernel: Uses multiple thread blocks per node for very large hubs, employing a parallel merge-sort or radix-sort approach depending on the data type.

5.2 Integration with ADDS

Once the edges are sorted, the ADDS SSSP kernel remains largely unchanged, but the order of its atomicMin operations on the distance array changes. Because the smaller weights are processed first, the distance values converge faster. This reduces the number of times a node is “re-enqueued” into the priority buckets, directly reducing the total work done by the GPU.

6. Performance Evaluation

6.1 Sorting Performance Comparison

I compared the Three-Tier Hybrid strategy against several baselines on a standard large-scale graph dataset.

Implementation	Runtime (ms)	Notes
CPU std::sort	40,679	Sequential processing of each CSR segment.
Naive GPU Sort	27,653	Simple global sort without tiering.
Three-Tier Hybrid GPU	8,796	Custom implementation described above.
Thrust Segmented Sort	7,691	Highly optimized library sort.

While the Thrust implementation is slightly faster (due to its highly optimized low-level primitives), the Three-Tier Hybrid GPU approach is significant because it allows for in-place sorting within the existing CSR structure without the overhead of creating auxiliary segment-key arrays required by Thrust’s `zip_iterator`. Furthermore, the hybrid strategy is more flexible for custom multi-key comparison functions.

6.2 SSSP Runtime Improvement

The primary goal was to improve SSSP performance on power-law graphs (RMAT). RMAT graphs are defined by parameters (a, b, c, d) . As a increases, the graph becomes more skewed (more power-law like).

Graph Case	Parameters	With PRO		Improvement
	(a, b, c)	No PRO (ms)	(ms)	
Balanced	$a = 0.30, b = 0.25, c = 0.25$	638.701	636.704	~0.3%

Graph Case	Parameters (a, b, c)	No PRO (ms)	With PRO (ms)	Improvement
Skewed	$a =$ $0.45, b =$ $0.25, c =$ 0.15	464.656	419.049	~9.8%
Highly Skewed	$a =$ $0.57, b =$ $0.19, c =$ 0.19	805.626	601.142	~ 25.4%

Analysis: The data clearly shows that the PRO optimization is highly effective for power-law distributions. In the $a = 0.57$ case, the SSSP runtime was reduced by over 200ms. This confirms the hypothesis that sorting edges by weight allows the asynchronous engine of ADDS to discover shorter paths earlier in the hub-dominated clusters, preventing the “propagation of suboptimal distances” that often plagues parallel SSSP.

7. Correctness and Performance Analysis

7.1 Correctness

The correctness of the sorting algorithm is guaranteed by the properties of the Bitonic Sort. Bitonic sort is a data-independent sorting network, meaning the sequence of comparisons is the same regardless of the data values. By ensuring that the comparison distance j is correctly decremented and that all threads within the warp or block synchronize at each stage, the algorithm is proven to sort correctly.

The correctness of the SSSP is unaffected by PRO. SSSP algorithms like ADDS or Bellman-Ford are robust to the order of edge relaxations; edge sorting is purely a performance heuristic that does not change the final shortest-path result.

7.2 Theoretical Complexity

- Warp/Block Sort: For a node of degree d , the Bitonic Sort complexity is $O(d \log^2 d)$.
- Parallelism: Each node is processed in parallel. In the best case (Tier 1), we have $O(V/32)$ warps working simultaneously.
- Global Memory Traffic: The Tier 3 hybrid strategy reduces the complexity of global memory access from $O(d \log^2 d)$ to $O(d \log(d/S) + d \log^2 S)$ where S is the shared memory size. This is a significant constant-factor improvement in bandwidth utilization.

8. Conclusion and Future Work

This project successfully implemented and integrated a Three-Tier Hybrid Sorting scheme into the ADDS SSSP framework. By leveraging specific GPU hardware features—Registers/Warp Shuffles for small degrees, Shared Memory for medium degrees, and a Hybrid Global approach for large hubs—the implementation achieves performance comparable to high-level libraries while maintaining a lower memory footprint.

The experimental results prove that the Property-driven Reordering (PRO) optimization is vital for processing power-law graphs. The 25% performance gain observed in highly skewed RMAT graphs validates the strategy of prioritizing edges with smaller weights to accelerate distance convergence.

Future Work:

- **Dynamic Tiering:** Currently, the degree ranges for tiers are fixed. Implementing a dynamic system that considers current GPU occupancy and shared memory availability could further optimize throughput.
- **Multi-GPU Sorting:** For graphs that exceed the memory of a single GPU, a distributed segmented sort using NVLink would be necessary.